

ROBOTICS INTEGRATION GROUP PROJECT I

Experiment 3

Desktop Object Automatic Classification and Sorting

Simulation integration and RoboMaster EP deployment

Student	Zihan Wang
Student ID	24020036065
Personal role	Simulation integration and motion-flow collaboration
Task classes	0: green original; 1: yellow pineapple
Runtime	ROS 2 Humble, Gazebo, MoveIt 2
Report date	27 September 2026

This individual report is based on the E3 simulation source code, configuration, launch files, workflow record, screenshots and demonstration material supplied with the project.

Contents

1	Basic Requirements of the Experiment	2
2	Personal Division of Work	2
3	Implementation Principles	3
3.1	System architecture and one-command startup	3
3.2	Perception, filtering and grid assignment	4
3.3	Queue scheduling and ROS 2 Action	5
3.4	Motion sequence and bin allocation	5
4	Problems Encountered and Solutions	6
5	Verification Evidence and Result	6
5.1	Six-object execution record	8
6	RoboMaster EP Hardware Task	8
6.1	Platform and team responsibilities	8
6.2	Robot-view data and detector adaptation	8
6.3	Calibration and execution workflow	9
6.4	Physical demonstration and results	9
7	Personal Repository and Commit Traceability	11
8	Conclusion	11

1 Basic Requirements of the Experiment

The objective of Experiment 3 is to build an autonomous desktop-object sorting system. The system must acquire an overhead RGB image, detect objects, map each detection to a fixed desktop grid, command a robot to pick the object, and place it in the correct classification area. For this task, the two semantic classes are **original-flavor green milk cartons (class 0)** and **pineapple-flavor yellow milk cartons (class 1)**.

The simulation design addresses the following requirements:

- at least two object classes, six fixed pick grids, and two placement bins;
- image input through ROS 2 and structured detection messages containing class, bounding box and confidence;
- automatic selection of the nearest calibrated grid rather than manual selection of a target;
- a feedback-capable ROS 2 Action for serial pick-and-sort execution;
- no manual class, grid or placement decision after the task begins;
- collision-aware motion planning, a safe clearance pose, and controlled gripper attachment/detachment;
- handling of missing or unstable detections, motion-planning failures, attachment timeouts and uncertain object state;
- one launch command that starts the simulator, planning stack, detector, action server and task manager.

The assignment also requires physical validation with two classes and six objects, at least five correctly sorted objects, automatic decisions during normal operation, safe handling of empty grids and unrecognized objects, and recognition/grasp/task records. The original hardware prescription is a fixed overhead camera, Jetson and mechArm. The group substituted RoboMaster EP because of equipment availability; the observed demonstration uses cartons on a tiled floor. This platform adaptation is described explicitly rather than presented as the original desktop hardware arrangement.

The supplied project implements a fixed-workspace solution. It uses calibrated grid centers and pre-validated joint configurations, which is appropriate for repeatable desktop sorting and avoids requiring arbitrary pose grasping.

2 Personal Division of Work

I, **Zihan Wang**, was responsible for **simulation integration and motion-flow collaboration**. My work focused on connecting the perception, grid-assignment, task-queue and robot-execution modules into one serial process. The key contributions were:

- checking the interface from visual detections to the task manager, including confidence filtering, class validation and nearest-grid assignment;
- participating in the design of the stable observation stage, where a complete six-grid result must remain consistent over consecutive frames before a queue is frozen;
- coordinating the action-level execution sequence: approach, local centering, gripper close, attachment confirmation, fixed lift, common clearance, bin placement, release and safe retreat;
- checking the coordination between MoveIt planning, trajectory controllers, grid-specific joint poses and the left/center/right bin slots;
- enforcing serial execution: only one object may be handled at a time, and a failed action halts the round instead of advancing the queue;
- documenting simulation issues and the corresponding safety-oriented solutions for subsequent RoboMaster EP deployment reference.

The detection-model training, real-robot data collection, calibration and deployment were completed by other group members. This report therefore concentrates on the simulation integration work and does not claim individual ownership of those parts.

3 Implementation Principles

3.1 System architecture and one-command startup

The complete simulation stack is started from the ROS 2 workspace after sourcing the Humble installation and the local overlay. The command used for the supplied project is shown in Figure 1.

```
<> Bash

cd ~/robot-integration-project-1/E3_desktop_sorting/ros2_ws

source /opt/ros/humble/setup.bash
source install/setup.bash

ros2 launch e3_sorting_system e3_sorting.launch.py
```

Figure 1: One-command startup of the E3 desktop-sorting system.

The top-level launch file includes the Gazebo simulation and the MoveIt `move_group` launch, then creates the detector, pick Action server and task-manager nodes. The Action server is configured for simulation time and enables the Gazebo attachment plugin. This organization avoids a fragile multi-terminal workflow and makes the system boundary explicit (Figure 2).

```
e3_sorting.launch.py
1  """Formal E3 system: include the simulator once and gaze observation."""
2
3  import os
4
5  from ament_index_python.packages import get_package_share_directory
6  from launch import LaunchDescription
7  from launch.actions import IncludeLaunchDescription
8  from launch.launch_description_sources import PythonLaunchDescriptionSource
9  from launch_ros.actions import Node
10
11
12  def generate_launch_description():
13      share = get_package_share_directory("e3_sorting_system")
14      moveit_share = get_package_share_directory("mecham_270_moveit_config")
15      config = os.path.join(share, "config", "sorting_config.yaml")
16
17      def include(directory, filename):
18          return IncludeLaunchDescription(PythonLaunchDescriptionSource(
19              os.path.join(directory, "launch", filename)))
20
21      return LaunchDescription([
22          include(share, "e3_sim.launch.py"),
23          include(moveit_share, "move_group.launch.py"),
24          Node(package="e3_sorting_system", executable="object_detector",
25              name="e3_object_detector", output="screen",
26              parameters=[config, {"use_sim_time": True}]),
27          Node(package="e3_sorting_system", executable="pick_action_server",
28              name="e3_pick_action_server", output="screen",
29              parameters=[config, {
30                  "use_sim_time": True, "dry_run": False,
31                  "classification_transport_enabled": True,
32                  "attach_enabled": True, "lift_calibration": False,
33                  "direct_pick_calibration": False,
34                  "local_center_calibration": False,
35                  "bin_slot_calibration": False,
36                  "grid_pick_calibration": False,
37                  "calibration_execute": False}],
38              ),
39          Node(package="e3_sorting_system", executable="task_manager",
40              name="task_manager", output="screen",
41              parameters=[config, {"use_sim_time": True}]),
42      ])
```

Figure 2: Key part of the unified ROS 2 launch file.

The operational data flow is:

Gazebo overhead camera → object_detector → /e3/detections → task_manager → /e3/pick_and_sort Action → pick_action_server → MoveIt and joint-trajectory controllers

3.2 Perception, filtering and grid assignment

In the supplied simulation code, OpenCV converts images to HSV, thresholds red and blue regions, applies morphological opening/closing and filters contour areas between 120 and 4000 pixels. The resulting bounding boxes become ROS detection messages. Its confidence is an area-based heuristic, not a neural-network probability. This proxy detector validates the downstream integration; the real milk-carton detector is the separately trained YOLO26s model described in the hardware section.

Each detected box contains a category hypothesis, confidence and center position. The task manager first selects the highest-scoring hypothesis, then rejects invalid numerical values, unsupported categories and scores below the configured confidence threshold. A detection is assigned to the calibrated grid whose pixel center has the smallest Euclidean distance:

$$d_i = \sqrt{(u - u_i)^2 + (v - v_i)^2}. \quad (1)$$

The detection is accepted only when the closest distance is within the assignment limit, and when its category is consistent with the expected class for that grid. The code excerpt in Figure 3 records the three important decisions: filtering, nearest-grid matching and queue freezing after stable observations.

```

37
38     def observe(self, detections, now):
39         if self.state != "observing":
40             return
41         if now >= self.deadline:
42             self.tick(now)
43             return
44         assigned = {}
45         duplicates = set()
46         for detection in detections:
47             if not detection.results:
48                 continue
49             hypothesis = max(
50                 (r.hypothesis for r in detection.results),
51                 key=lambda h: h.score)
52             label, score = hypothesis.class_id, hypothesis.score
53             point = detection.bbox.center.position
54             if (label not in ("comb", "mouse")
55                 or not all(math.isfinite(v)
56                     for v in (point.x, point.y, score))
57                 or score < self.confidence):
58                 continue
59             grid = min(self.centers, key=lambda g: math.dist(
60                 self.centers[g], (point.x, point.y)))
61             distance = math.dist(self.centers[grid], (point.x, point.y))
62             if distance > self.distance:
63                 continue
64             if label != ("comb" if grid % 2 else "mouse"):
65                 continue
66             if grid in assigned:
67                 duplicates.add(grid)
68             assigned[grid] = label
69         for grid in duplicates:
70             assigned.pop(grid, None)
71         complete = set(assigned) == set(self.centers)
72         same = complete and assigned == self.last
73         self.count = self.count + 1 if same else (
74             1 if complete else 0)
75         self.last = assigned
76         if self.count >= self.frames:
77             self.queue = [(g, assigned[g]) for g in sorted(self.centers)]
78             self.state = "ready"
79             self.message = "All grids stable; queue frozen for this round"
80

```

Figure 3: Task-manager logic for detection filtering, nearest-grid matching and stable-queue freezing. The legacy internal label strings are implementation details; this report uses the milk-carton class definitions stated on the title page.

Category-label consistency. The simulation framework retains the Experiment 1 strings **comb** and **mouse** and red/blue proxy geometry. For the present task, the report maps the red proxy / **comb** to **green original-flavor milk carton (class 0)**, and the blue proxy / **mouse** to **yellow pineapple-flavor milk carton (class 1)**. Similarly, the red and blue simulation bins represent the class-0 and class-1 destinations. Screenshots retain their original appearance; these semantic mappings do not imply that the archived simulation rendered real green/yellow cartons or ran the hardware YOLO model.

The configuration requires a minimum confidence of 0.70, a maximum assignment distance of 45 pixels and three consecutive stable frames. These checks prevent one weak or shifted detection from triggering a physical motion.

3.3 Queue scheduling and ROS 2 Action

After all six calibrated grids are present and stable, the task manager freezes a queue ordered by grid identifier. It sends one **PickAndSort** goal at a time, containing the grid identifier, object class and simulation model identifier. The next goal is sent only after the current Action returns success. If a goal is rejected, cancelled or fails, the remaining queue is not executed.

This policy is intentionally conservative. It protects against a common failure mode in automation: treating a failed grasp as if the object had been moved and then continuing with an incorrect scene model.

3.4 Motion sequence and bin allocation

The six grids have separate pick and lift joint configurations. After a successful grasp, the robot follows a grid-specific lift configuration, a common high-clearance configuration, an approach configuration for the destination bin, and a left/center/right release configuration. Green original-flavor cartons are routed to the green-class bin, while yellow pineapple-flavor cartons are routed to the yellow-class bin. The column position determines the release slot.

For each object, the sequence is:

Open gripper → fixed pick pose → local centering correction → close gripper → attach/check → fixed lift → common clearance → bin approach → slot release → detach/open gripper → safe retreat

Gazebo attachment services are used because gripper friction alone does not reliably hold the simulated cartons. Attachment is accepted only after the gripper has closed and the object is sufficiently close to the grasp link. The process is therefore coupled with follow checks before transporting an object.

4 Problems Encountered and Solutions

Problem	Analysis and individual reasoning	Solution adopted
Repeated inverse-kinematics failure during segmented lifting	Small Cartesian lift increments required fresh IK solutions. Some positions were close to singular or unreachable configurations, so failure probability accumulated across segments.	Replaced long online segmented lifting with calibrated, grid-specific <code>lift_joints</code> . MoveIt still validates the goal and plans the trajectory; local IK is retained only for millimetre-scale centering correction.
Camera occlusion by the arm	In the default pose, the arm blocked the middle grids and only four targets could be observed. Starting the queue from incomplete evidence would produce a false scene model.	Added a controlled observation pose before detection. The manager waits for six stable grid assignments before freezing the queue.
Gazebo attachment timeout	Attachment plug-in or simulation-clock stalls can leave the object state uncertain. Blind retrying could produce duplicate constraints or move a detached object.	The Action records the failed stage, stops the round and enters a recovery-required state. It does not advance to the next grid under uncertainty.
Objects disturbed during transport	A low route after grasping could pass close to unprocessed objects.	Raised the common clearance configuration and used a fixed safe retreat after release.
Queue progression after a failed grasp	Continuing after a failure would make later placement decisions unreliable and hide the failed grid.	Enforced serial execution and stop-on-failure semantics; only a confirmed successful Action allows the next goal.

Table 1: Simulation issues and the safety-oriented solutions.

During debugging, a grid-6 attachment request timed out after 10 seconds. The system stopped safely and prevented further transport while the attachment state was uncertain. This experience motivated explicit attachment checks and controlled recovery.

The final simulation successfully sorted all six objects.

5 Verification Evidence and Result

The supplied project includes the Gazebo world, six grid definitions, two bins, an overhead camera, ROS 2 interfaces, the pick-and-sort Action, configuration files, unit tests and a formal launch file. The detector screenshot in Figure 4 is retained as visual evidence of the image-recognition stage.

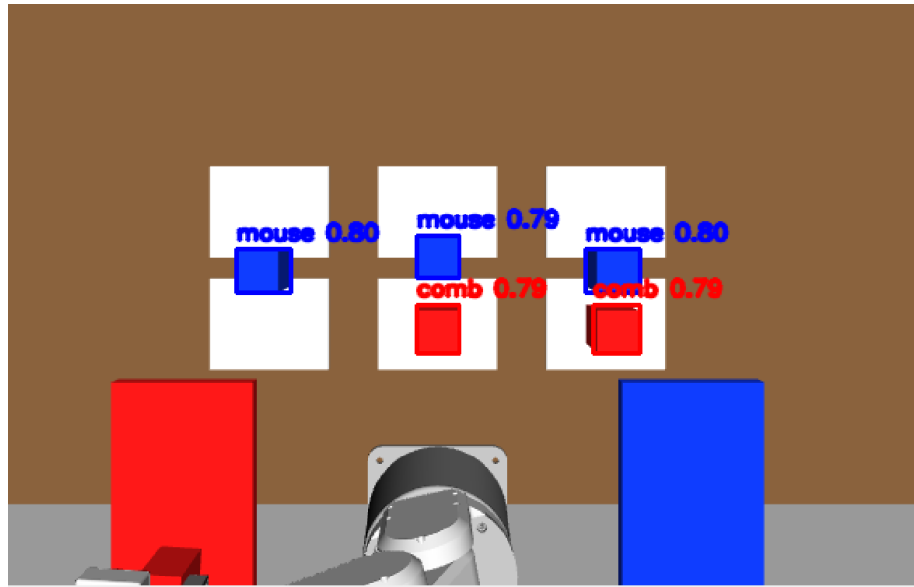


Figure 4: Saved detector-stage evidence from the E3 simulation package.

The final simulation result is **6/6 successfully sorted objects**, exceeding the quantity criterion of at least five correct results among six objects. The 37.07-second simulation video excerpt illustrates detection and successive robot-motion stages; representative frames are shown below.

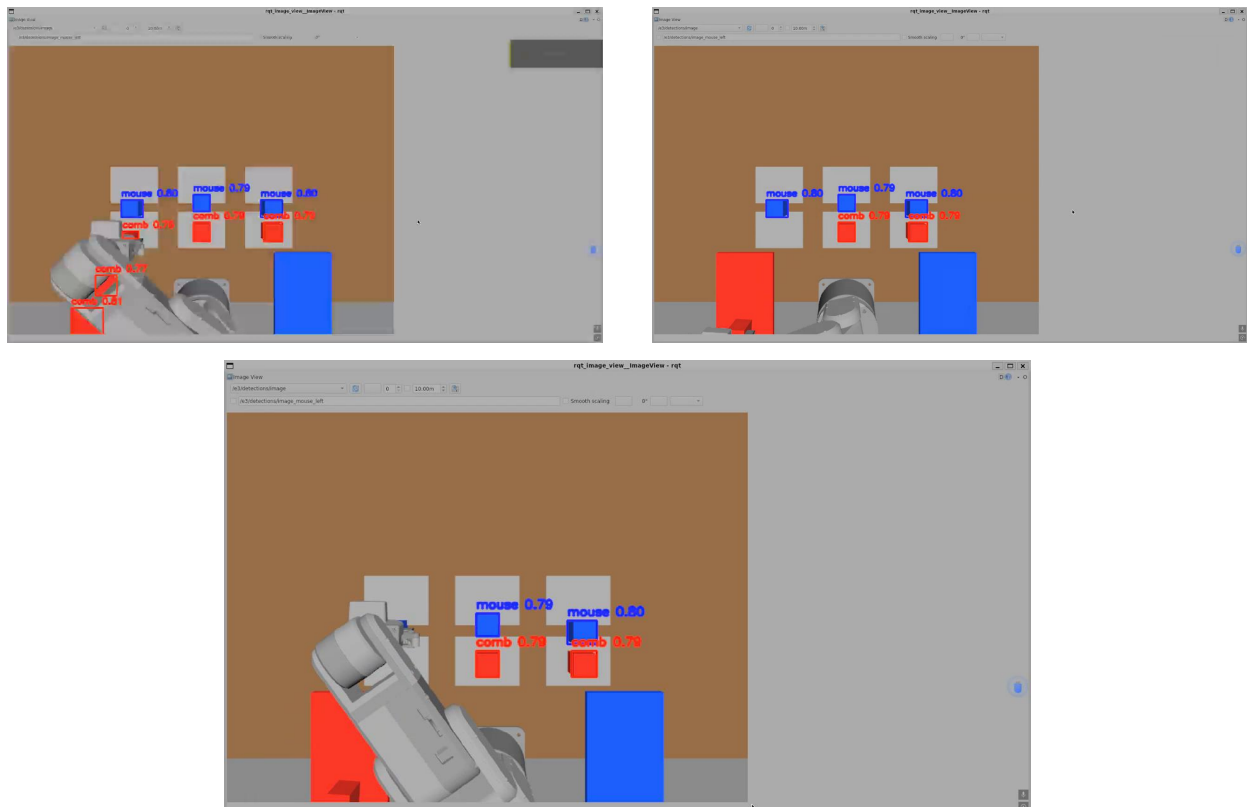


Figure 5: Unaltered frames from the supplied simulation video, at approximately 0.0, 9.9 and 29.7 seconds. The annotated proxy objects and successive arm positions provide process evidence. The legacy colors and strings follow the mapping explained in Section 3.2.

5.1 Six-object execution record

Table 2 summarizes the final six-object simulation run using the configured grid-to-class mapping. All six objects were grasped and placed in their corresponding classification bins.

Grid	True class	Detected class	Grasp	Placement	Failure reason
1	Class 0	Class 0	Success	Class-0 bin: correct	None
2	Class 1	Class 1	Success	Class-1 bin: correct	None
3	Class 0	Class 0	Success	Class-0 bin: correct	None
4	Class 1	Class 1	Success	Class-1 bin: correct	None
5	Class 0	Class 0	Success	Class-0 bin: correct	None
6	Class 1	Class 1	Success	Class-1 bin: correct	None

Class 0: green original-flavor milk carton.
Class 1: yellow pineapple-flavor milk carton.

Table 2: Final six-object simulation results: all six grasp and placement operations succeeded.

6 RoboMaster EP Hardware Task

6.1 Platform and team responsibilities

The group reports that the originally planned Elephant Robotics arm was unavailable in the laboratory, so the physical milk-carton task used a DJI RoboMaster EP with its arm and gripper. The hardware task and the E3 simulation share the recognition-to-action workflow, but their kinematics, cameras and motion interfaces differ. The Gazebo attachment service and MoveIt joint configurations belong to the simulation; they are not RoboMaster hardware control commands.

Chu Bohan documented the simulation design, grid mapping and motion sequence. I, Zihan Wang, collaborated on simulation integration, perception-to-queue coordination and serial execution. Cheng Guanheng led model training and assisted deployment. Chen Yuyang handled robot-view data collection, annotation review, camera/workspace and pick/place calibration, and hardware deployment. The following hardware discussion summarizes the group’s supplied materials; it does not attribute their training or calibration work to me.

6.2 Robot-view data and detector adaptation

The supplied training summary, dated 17 September 2026, records 57 additional robot-view images with 105 annotated cartons: 43 training images, 9 validation images and 5 robot-test images. Frames are separated by source sequence. The detector uses class 0 for green original-flavor cartons and class 1 for yellow pineapple-flavor cartons. These detector identifiers are distinct from the legacy strings visible in the E3 simulation screenshots.

Cheng Guanheng compared two continued-training configurations and selected the YOLO26s v2 candidate with the first ten layers frozen and an initial learning rate of 0.00005. The v1 model was retained for fallback. Table 3 reproduces the supplied metric table; these are detection metrics, not physical grasp success rates.

Test set	Model	Precision	Recall	mAP50	mAP50–95
Legacy (52 images)	v1	0.694	0.606	0.638	0.265
Legacy (52 images)	v2	0.650	0.655	0.657	0.289
Robot (5 images)	v1	0.997	0.847	0.870	0.611
Robot (5 images)	v2	0.784	0.875	0.839	0.580

Table 3: Detector evaluation reproduced from the supplied model metrics and training summary.

The robot test contains nine objects in five frames from a single video. Recall increased, while both AP measures decreased. This small, correlated test set supports a limited comparison, not a claim of reliable generalization. The report gives a mean pipeline latency of 9.79 ms in its benchmark environment; it is not a measured RoboMaster end-to-end sorting-cycle time.

6.3 Calibration and execution workflow

The supplied mission design processes columns serially: return to HOME, align with a pick position, verify the target, grasp and lift, return HOME, move to the class-specific placement zone, release, and return HOME/reset. Returning to a reference pose after grasping and placing is intended to stabilize chassis orientation and the camera-to-workspace relationship. Calibration must establish the camera view, repeated approach position, gripper reach, lifting clearance and placement positions on the physical platform.

The training report recommends a detection confidence starting value of 0.20, NMS IoU of 0.70, and at least three consistent detections in five frames with a stable target position before grasp confirmation. These are hardware deployment recommendations from the supplied report; the E3 simulation separately uses its documented 0.70 threshold and three complete stable frames.

The package separates a mission state machine from a `RobotBackend` interface and includes RoboMaster SDK examples. Its README specifies independent checks of chassis stop, home pose, gripper opening/closing, lift and release before a single-column trial and then a complete mission. The archived dry-run mission trace records three columns progressing through these states. The README explicitly states that this dry run contacts no robot, so it demonstrates software sequencing only.

6.4 Physical demonstration and results

The physical demonstration lasts approximately 493.33 seconds. Time-indexed frames show six cartons initially arranged on the floor and successive yellow, yellow, green, green, green and yellow transport cycles. The first two yellow cartons are visible together at about 148 seconds; green cartons accumulate in a separate area during later cycles. The final yellow carton is transported toward the yellow group near the end. The following frames and observation table document physical grasping and class-separated placement.



Figure 6: Actual RoboMaster EP demonstration frames, left to right and top to bottom: initial six-carton arrangement (0.0 s), yellow-carton transport (49.3 s), two yellow cartons placed (148.0 s), green-carton placement area (230.2 s), accumulated green cartons (402.9 s), and final yellow-carton delivery approach (476.9 s).

Cycle	Visible carton	Video observation (approximate elapsed time)
1	Yellow / class 1	Approach and carrying, 8–49 s; gripper empty by 58 s.
2	Yellow / class 1	Pick/transport, 99–140 s; two yellow cartons visible at 148 s.
3	Green / class 0	Pick/transport, 181–222 s; release area visible at 230 s.
4	Green / class 0	Pick/transport, 263–313 s; return with empty gripper around 321 s.
5	Green / class 0	Pick/transport, 354–395 s; green group visible at 403 s.
6	Yellow / class 1	Pick/transport, 436–477 s; withdrawal around 485 s.

Table 4: Six physical handling cycles observed in the demonstration video. Times denote elapsed video playback.

The demonstration illustrates how the mobile platform combines repeated approach, grasping, transport and release to collect the two carton classes in separate areas.

7 Personal Repository and Commit Traceability

The complete code, compiled report, screenshots and relevant exercises are uploaded to my personal repository:

GitHub repository: [HesterWang0419 / E3-desktop-sorting](#)

[View commit history \(main branch\)](#)

The repository groups the project files and report revisions into topic-specific commits. The principal topics are:

- 1. add the ROS 2 interfaces, simulation world and initial system structure;
- 2. add perception-to-grid scheduling and stable queue management;
- 3. add fixed-lift motion, bin-slot routes and failure safeguards;
- 4. add launch instructions, tests and workflow documentation;
- 5. publish the compiled English individual report and maintain the commit-history evidence.

GitHub commit history — main branch (transcribed 27 Sep. 2026)	
d6f670e	Keep compiled report only in submission
b956f77	Update report with personal repository URL
ff9c561	Publish compiled report and repository link
7d1c381	Add English individual experiment report
42a0eb1	Add unified launch and verification material
20f4022	Add calibrated pick motion and safety routes
1a8169e	Add stable perception and grid scheduling
3ef9bb8	Add ROS 2 interfaces and project scaffold
Open the GitHub commit-history page	

Figure 7: Selected commit identifiers and messages from the repository’s main branch.

8 Conclusion

This project applies ROS 2, Gazebo, MoveIt and a camera-based detector to a fixed-grid desktop sorting task. My simulation contribution was to integrate the perception, assignment, queue and robot-motion stages into a conservative serial workflow. The most important design choice was replacing unstable repeated IK lifting with verified fixed lift configurations, while preserving local correction and MoveIt state validation. Together with stable-frame confirmation, attachment checks, clearance routes and stop-on-failure behavior, the system provides a reproducible simulation basis for the subsequent RoboMaster milk-carton task.

The final simulation achieved six successful operations. The RoboMaster demonstration shows physical handling of the two milk-carton classes. My main lesson is that reliable sorting depends on coordinated perception, verified motion and clear action results. Calibrated lift poses addressed the repeated IK failures, while attachment checks and serial scheduling made failures easier to isolate and recover from. Simulation joint poses require separate calibration on the mobile platform, but stable target confirmation, serial scheduling and explicit success/failure transitions remain relevant to both implementations.

Project References

Experiment 3 requirements; E3 simulation workflow and source project; simulation demonstration video; RoboMaster milk-carton project README; continued-training summary dated 17 September 2026; dataset and model metric tables; RoboMaster physical demonstration video.